

Examination Timetabling Problem

Pham Duy Nguyen - 202416730

June 8, 2025

Abstract

The Examination Timetabling Problem (ETP) is a well-known NP-hard combinatorial optimization problem with significant practical relevance in educational institutions. It involves assigning a set of exams to available time slots and rooms while satisfying various hard and soft constraints. This study investigates five solution techniques: a Constraint Programming (CP) model, a Mixed Integer Programming (MIP) model, and two heuristic-based methods. Experimental results demonstrate that the heuristic-based approach consistently outperforms CP and MIP in terms of solution quality and computational efficiency. A comprehensive analysis of the strengths and limitations of each method is presented, along with discussions on possible improvements and future research directions. The findings contribute to the advancement of ETP-solving methodologies and provide actionable insights for practitioners aiming to optimize exam scheduling and related packing problems.

1 Introduction and Objectives

There are N classes $1, 2, \dots, N$ that need to be scheduled for the final exam. Each class must be assigned to a time-slot and a room. There are M rooms $1, 2, \dots, M$ that can be used for scheduling the exam. Each room i has capacity $c(i)$ (number of places in the room). Each day is divided into 4 slots $1, 2, 3, 4$. Each class i has number of students $d(i)$ ($i = 1, \dots, N$). Among N classes, there are K pairs of classes (i, j) in which class i and class j have the same students participating in the exam. It means that these 2 classes cannot be scheduled in the same time-slot. Compute the exam time-table such that the number of days used is minimal.

The data format

- First line: N and M
- Second lines: $d_1, d_2, \dots, d_N$
- Third lines: $c_1, c_2, \dots, c_M$
- Fourth line: K
- Last K lines: i and j (2 classes having a same student registered, these classes cannot be scheduled in the same slot)

Generated Data

The input size is denoted by $(N \times M)$.

- 15 tests:
 $(5 \times 2), (6 \times 2), (7 \times 3), (8 \times 3), (9 \times 4),$
 $(10 \times 3), (11 \times 4), (12 \times 4), (13 \times 4), (14 \times 4),$
 $(15 \times 4), (16 \times 4), (17 \times 4), (18 \times 5), (19 \times 5)$
- 10 tests:
 $(50 \times 10), (51 \times 10), (52 \times 11), (53 \times 11), (54 \times 11),$
 $(55 \times 11), (56 \times 12), (57 \times 12), (58 \times 12), (59 \times 13)$
- 10 tests:
 $(100 \times 10), (200 \times 10), (300 \times 10), (400 \times 10), (500 \times 10),$
 $(600 \times 11), (700 \times 11), (800 \times 11), (900 \times 10), (1000 \times 10)$
- 3 tests mainly for heuristic testing:
 $(1000 \times 20), (2000 \times 20), (3000 \times 20)$

Each test case is randomly generated as follows:

- Each class has a number of students between 20 and 70.
- Let S be the largest number of students in any class. Each room has a capacity between $S/2$ and $3S/2$.
- The number of pairs of classes that share at least one student is at most $N(N+1)/4$.

2 Modeling the Problem

In the model, let:

- N : the number of classes
- M : the number of rooms
- d_i : the number of students in class i
- c_j : the capacity of room j
- P : the set of conflict pairs (i.e., pairs of classes that share at least one student)

2.1 CP Model

2.1.1 Decision Variables

- $slot_i \in \{1, \dots, N\}$: time-slot assigned to class i
- $room_i \in \{1, \dots, M\}$: room assigned to class i

2.1.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$d_i > c_j \Rightarrow \text{room}_i \neq j$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\text{slot}_i \neq \text{slot}_j, \forall (i, j) \in P$$

- Only one class per room per time-slot:

$$\text{slot}_i = \text{slot}_j \Rightarrow \text{room}_i \neq \text{room}_j$$

2.1.3 Objective Function

Minimize the number of time-slots used:

$$\min \left(\max_{i=1}^N \text{slot}_i \right)$$

2.2 MIP Model

2.2.1 Decision Variables

- $x_{i,s,r} \in \{0, 1\}$: binary variable, equals 1 if class i is assigned to time-slot s and room r , otherwise 0
- $D \in [1, N]$: the maximum time-slot index used (i.e., latest slot with an exam scheduled)

2.2.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$x_{i,s,r} = 0 \text{ if } c_j < d_i$$

- Each class is scheduled exactly once:

$$\sum_{s=1}^N \sum_{r=1}^M x_{i,s,r} = 1, \forall i \in \{1, \dots, N\}$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\sum_{\text{valid } r} (x_{i,s,r} + x_{j,s,r}) \leq 1, \forall (i, j) \in P, \forall s \in \{1, \dots, S\}$$

- Only one class per room per time-slot:

$$\sum_{i=1}^N x_{i,s,r} \leq 1, \forall s \in \{1, \dots, S\}, \forall r \in \{1, \dots, M\}$$

- Tracking the latest time-slot used:

$$\sum_{s=1}^N \sum_{r=1}^M (s+1) \cdot x_{i,s,r} \leq D, \forall i \in \{1, \dots, N\}$$

2.2.3 Objective Function

Minimize the number of time-slots used:

$$\min D$$

2.3 Heuristic

2.3.1 Overview

This section presents two heuristic algorithms used to solve ETP, a **Greedy Algorithm** and a **Tabu Search Algorithm**. The Greedy Algorithm offers faster solutions, while the Tabu Search prioritizes improving solution quality.

In this report, the term "cost" represents the number of slots required to schedule all exams.

2.3.2 Sorting Input

Before applying either algorithm, we preprocess the input:

- **Room Fit Filtering:** For each class, only rooms with enough capacity are taken into consideration.
- **Class Order Randomization:** Classes are shuffled randomly before scheduling.

2.3.3 Choosing Room for Classes

- **Greedy Algorithm:**
 - Classes are placed in the first room and time-slot that satisfy the conflict and capacity constraints.
 - The search prioritizes lower-indexed slots to minimize the maximum slot used, which also minimizes the number of days.
- **Tabu Search Algorithm:**
 - Starts from either a Greedy or Random initial solution.
 - Moves high-slot classes to earlier satisfying slots using a neighborhood search.
 - Moves are restricted by a *tabu list* to prevent reversing previous moves and find new settings.
 - A cost function judges each settings based on the maximum slot used.

2.3.4 Process Demonstration

Greedy Algorithm

1. Iterate through each class (based on randomized order).
2. For each class, scan all rooms with enough capacity.
3. For each such room, scan time-slots to find the earliest satisfying slot where:
 - No conflicting class is scheduled.
 - Room is free.
4. Assign class to that room and time-slot, update room usage and conflict tracking.
5. Repeat across multiple shuffles (e.g., 5000 times) and choose the best result (lowest number of days).

Tabu Search Algorithm

1. Generate an initial solution using Greedy or Random assignment.
2. Find classes in the latest-used time-slots.
3. Find moves that reassign those classes to earlier satisfying slots and rooms.
4. Track recent moves in a *tabu list* to prevent reversal.
5. Update the current solution if a better one (i.e., lower day count) is found.
6. Continue for a fixed number of times or until no improvement is found for a number of steps.

2.3.5 Pseudo-code

The following pseudo-codes describe the implementation of the Greedy and Tabu Search Algorithm used in the proposed approach to solve the ETP.

The pseudo-codes can serve as a guideline for implementing the algorithms in the proposed approach and can be used as a starting point for further optimizations.

Algorithm 1: Greedy

```
For each shuffle iteration:
  Randomize class order.
  For each class:
    For each feasible room:
      For each time-slot:
        If room is free and no conflict:
          Assign class to room and slot.
          Update conflict and room usage.
          Break.
  Compute number of days used.
  If current solution is better:
    Save as best solution.
```

Algorithm 2: Tabu Search

```
Initialize solution (Greedy or Random).
Set best solution = initial.
Repeat until max iterations or no improvement:
  Identify classes in latest-used slot.
  For each such class:
    For each earlier slot and feasible room:
      If move is not tabu or improves best:
        Evaluate resulting configuration.
        If better than current best:
          Update best solution.
  Apply best move.
  Add move to tabu list.
```

3 Result and Analysis

3.1 Result

3.1.1 CP model and MIP model solver

Exact Solution

Input sizes		CP		MIP	
N	M	f	t(ms)	f	t(ms)
5	2	5	7	5	35
6	2	3	12	3	85
7	3	3	18	3	162
8	3	3	17	3	278
9	4	4	27	4	123
10	3	4	26	4	164
11	4	6	31	6	432
12	4	4	43	4	577
13	4	5	142	5	902
14	4	8	93	8	1334
15	4	9	72	9	28636
16	4	8	173	8	21724
17	4	10	188	10	301973
18	5	7	189	7	303064
19	5	8	13841	8	302770
50	10	5	7345	5	321161
51	10	7	300154	9	325693
52	11	8	300164	15	325189
53	11	14	300537	15	315767

Table 1: Results only exact solution of CP solver and MIP solver

All Solution

As a result of the large size and intricate nature of the optimization problems under consideration, we were only able to obtain feasible solutions for test instances with input size smaller than or equal to (53×10) for the CP solver, and (16×4) for the MIP solver, within a time limit of 300 seconds. This indicates that the optimization problems are highly complex and computationally demanding, requiring significant computational resources to solve. To ensure the reliability of our findings, we conducted ten runs of each solver and computed the average results, thereby mitigating the potential impact of random variations and ensuring the robustness of our numerical experiments.

Input sizes		CP				MIP			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	12	5	5	5	77
6	2	3	3	3	13	3	3	3	82
7	3	3	3	3	15	3	3	3	294
8	3	3	3	3	20	3	3	3	261
9	4	4	4	4	30	4	4	4	103
10	3	4	4	4	26	4	4	4	153
11	4	6	6	6	34	6	6	6	407
12	4	4	4	4	85	4	4	4	538
13	4	5	5	5	44	5	5	5	1086
14	4	8	8	8	97	8	8	8	1262
15	4	9	9	9	94	9	9	9	25634
16	4	8	8	8	100	8	8	8	22657
17	4	10	10	10	157	10	10	10	303677
18	5	7	7	7	170	7	8	8	301623
19	5	8	8	8	12577	8	8	8	304671
50	10	5	5	5	2162	5	5	5	318973
51	10	7	8	7.33	300043	9	9	9	320002
52	11	8	9	8.33	300049	15	16	15	322745
53	11	14	14	14	1654	15	15	15	315434
54	11	9	11	10	300043	9	11	10	314809
55	11	14	14	14	300311	15	15	15	311737
56	12	9	10	9.67	300179	13	15	14	326746
57	12	9	12	11	300181	18	18	18	336635
58	12	9	11	10.33	300191	23	27	25	335098
59	13	9	9	9	300356	17	17	17	342411
100	10	18	18	18	300534	F	F	F	F
200	10	184	210	195.33	301983	F	F	F	F
300	10	F	F	F	F	F	F	F	F
400	10	F	F	F	F	F	F	F	F
500	10	F	F	F	F	F	F	F	F
600	11	F	F	F	F	F	F	F	F
700	11	F	F	F	F	F	F	F	F
800	11	F	F	F	F	F	F	F	F
900	10	F	F	F	F	F	F	F	F
1000	10	F	F	F	F	F	F	F	F
1000	20	F	F	F	F	F	F	F	F
2000	20	F	F	F	F	F	F	F	F
3000	20	F	F	F	F	F	F	F	F

Table 2: Average results of CP solver and MIP solver

3.1.2 Heuristic solver

Input sizes		Greedy				Tabu Search			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	16	5	5	5	1723
6	2	3	3	3	15	3	4	3.67	2830
7	3	3	3	3	15	4	4	4	3402
8	3	3	3	3	15	3	3	3	4863
9	4	4	4	4	15	4	5	4.67	3486
10	3	4	4	4	15	5	6	5.67	3849
11	4	6	6	6	15	7	7	7	3234
12	4	4	4	4	16	4	6	5	5668
13	4	5	5	5	15	5	6	5.33	3628
14	4	8	8	8	16	8	8	8	2622
15	4	9	9	9	16	9	11	10	3793
16	4	8	8	8	16	8	9	8.67	3969
17	4	10	10	10	16	10	11	10.33	5001
18	5	7	7	7	16	8	10	9.33	6178
19	5	8	8	8	16	9	10	9.33	6036
50	10	8	9	8.33	18	10	11	10.33	14055
51	10	8	9	8.67	19	10	10	10	15318
52	11	9	10	9.33	18	11	11	11	16824
53	11	14	14	14	18	14	15	14.67	10448
54	11	11	11	11	18	11	13	12.33	10046
55	11	14	15	14.67	18	18	19	18.33	6792
56	12	10	11	10.67	18	11	13	12	14171
57	12	10	11	10.67	18	11	13	12	17858
58	12	11	11	11	18	12	13	12.33	12053
59	13	10	11	10.67	19	12	14	13	19433
100	10	20	20	20	18	21	23	22.33	9674
200	10	32	33	32.67	20	34	35	34.67	15611
300	10	50	50	50	22	52	53	52.33	20197
400	10	64	65	64.33	25	65	68	66.33	21616
500	10	74	75	74.67	29	76	80	78	29053
600	11	90	91	90.67	33	93	97	95	27099
700	11	91	92	91.67	55	94	98	95.33	49114
800	11	158	159	158.67	36	162	163	162.67	28225
900	10	128	129	128.67	51	130	133	132	37970
1000	10	180	181	180.67	40	184	187	185.33	32514
1000	20	122	122	122	118	126	127	126.67	65338
2000	20	177	178	177.67	381	181	183	182	167234
3000	20	175	176	175.33	259	500	501	500.33	198417

Table 3: Average results of Heuristic solver

3.2 Analysis

3.2.1 Exact Solution

Our evaluation of the accuracy of each algorithm revealed that CP and MIP were capable of generating exact solutions for a subset of test cases:

- CP was able to produce exact solutions for tests with sizes:
 - (5×2) , (6×2) , (7×3) , (8×3) , (9×4) , (10×3) , (11×4) , (12×4) , (13×4) , (14×4) , (15×4) , (16×4) , (17×4) , (18×5) , (19×5) , (50×10) , (51×10) , (52×11) , (53×11)
- MIP was only able to produce exact solutions for tests with sizes:
 - (5×2) , (6×2) , (7×3) , (8×3) , (9×4) , (10×3) , (11×4) , (12×4) , (13×4) , (14×4) , (15×4) , (16×4)

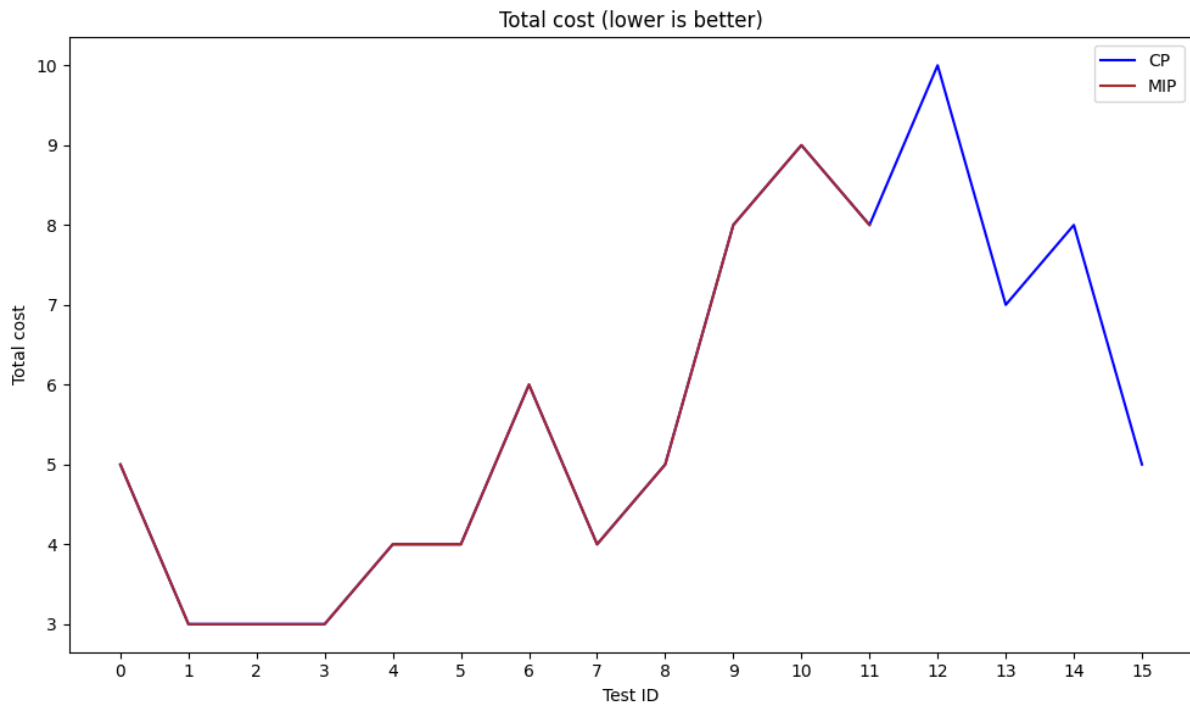


Figure 1: Results compare only exact solutions

3.2.2 All Solution

Our analysis of the solutions generated by four distinct algorithms revealed the following:

- CP is unable to handle datasets larger than (200×10) .
- MIP is unable to handle datasets larger than (59×13) .
- Both Greedy and TS are capable of handling all test cases.

In terms of total cost, our findings indicate:

- MIP performs the worst.
- CP and MIP generate nearly equivalent results on small test cases.
- Greedy and TS demonstrate very similar trends and clearly outperform CP and MIP in terms of total cost for test IDs ≥ 25 (i.e., data sizes larger than (200×10)).

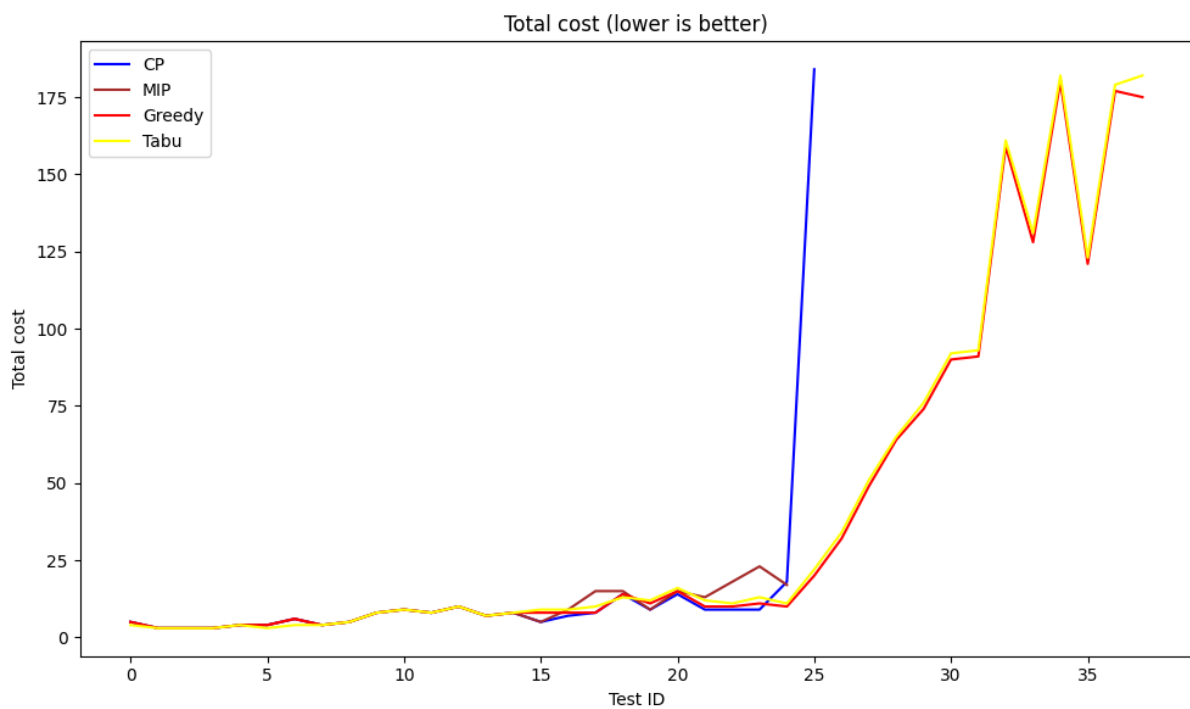


Figure 2: Results compare all solutions

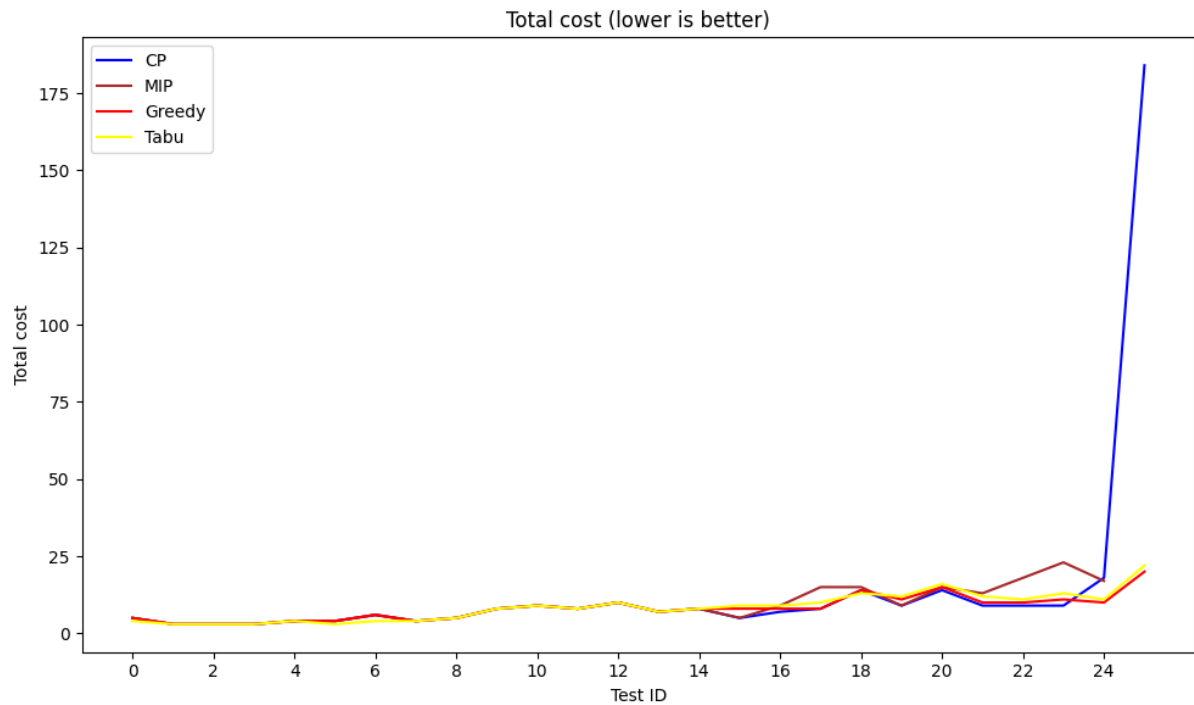


Figure 3: Results compare all solutions of the first 25 tests

3.2.3 Runtime

The running time of each algorithm was also evaluated, and our findings indicate that:

- MIP reaches the time limit of 300 seconds for all tests with size $\geq (17 \times 4)$.
- CP reaches the time limit of 300 seconds for all tests with size $\geq (51 \times 10)$.
- TS showed considerably short runtime for all tests with size $\leq (900 \times 10)$, but revealed to be less efficient for larger tests.
- Greedy showed shortest runtime among four demonstrated algorithms, with every test completed in under 1 second, even for the largest test with size (3000×20) .

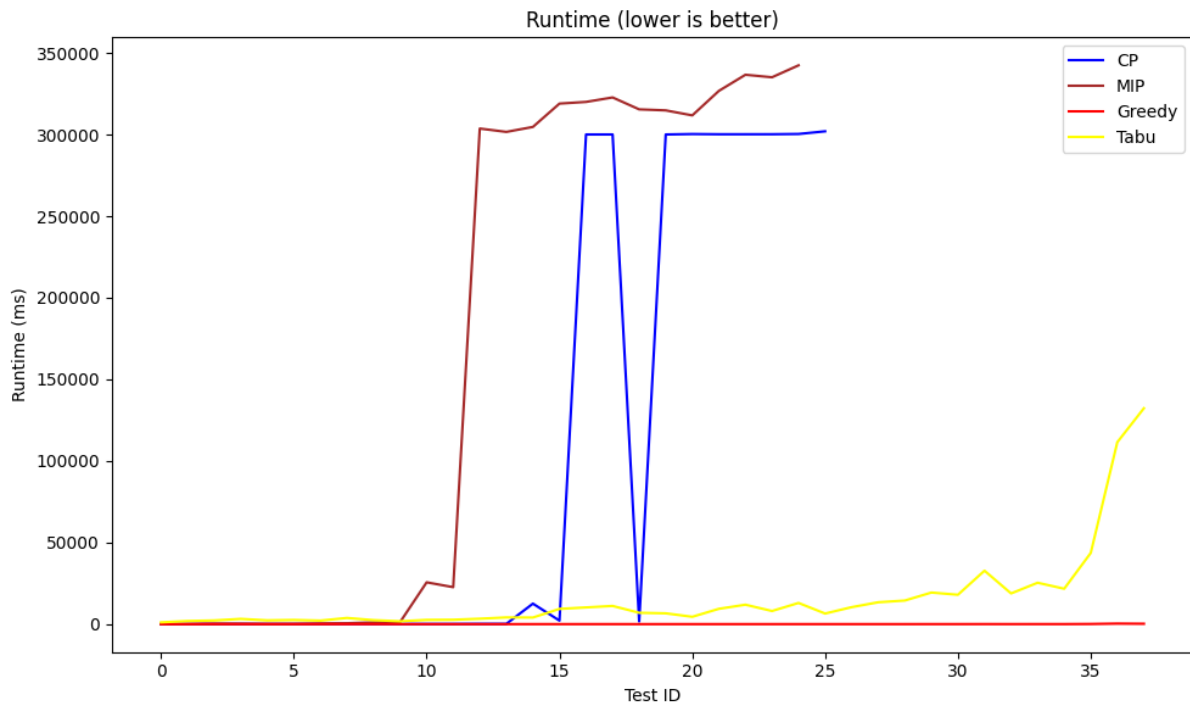


Figure 4: Runtime comparison of all algorithms

3.2.4 Summary

From our analysis, we conclude:

- The MIP solver consistently performed the worst in terms of computational cost and execution time.
- The CP solver was superior to MIP but struggled with scalability beyond moderate-sized instances.
- The heuristic approach proved to be the best, handling large datasets efficiently and generating optimal or near-optimal results.

These findings reinforce the practical applicability of heuristics for large-scale optimization problems, particularly for complex bin-packing scenarios.

4 Limitations and Future Research

Our analysis also revealed some limitations and potential avenues for future research.

Firstly, our study only considered a specific set of test cases, and the impact of varying problem sizes and parameter settings was not investigated. Therefore, future research could focus on evaluating the performance of the algorithms on a wider range of test cases and exploring the impact of different parameter settings.

Additionally, future research could focus on optimizing the performance of the heuristic algorithm while maintaining its accuracy and efficiency, or exploring the potential for hybrid algorithms that combine elements of the CP, MIP, and heuristic models to achieve even better performance.

Furthermore, our study only focused on the theoretical aspects of the optimization problem and did not consider the practical implications of implementing the algorithms in real-world scenarios. Thus, future research could investigate the potential for implementing these algorithms in various domains and evaluating their performance in practical settings.

In summary, our study provides insights into the performance of different algorithms for the optimization problem, highlights their limitations, and identifies potential areas for further research.

5 Visualization

In terms of visualization, we generated figures to display the results of the CP solver and the Heuristic solver. However, due to the MIP solver's long runtime and poor performance, we did not generate any figures for it. Therefore, it is not necessary to incorporate MIP solver results in the figures.

The figures comparing the results of the CP and Heuristic solvers are presented below:

Slot 1
 Slot 2
 Slot 3
 Slot 4

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

Table 4: CP solver

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

Table 5: Greedy solver

Class	Room 1	Room 2	Room 3
Class 1			
Class 2			
Class 3			
Class 4			
Class 5			
Class 6			
Class 7			
Class 8			
Class 9			
Class 10			

Table 6: TS solver

Figure 4: Result for test size (10×3)

References

- [1] E. S. K. Siew, S. N. Sze, S. L. Goh, G. Kendall, N. R. Sabar and S. Abdullah, *A Survey of Solution Methodologies for Exam Timetabling Problems*, IEEE Access, 2024.
- [2] Pillay, N., Qu, R., *Examination Timetabling Problems*, In: Hyper-Heuristics: Theory and Applications. Natural Computing Series, Springer, Cham, 2018.
- [3] Dezhen Zhang, Saijun Guo, Weishi Zhang and Shujuan Yan, *A novel greedy heuristic algorithm for university course timetabling problem*, Institute of Electrical and Electronics Engineers, 2014.
- [4] Zhipeng Lü, Jin-Kao Hao, *Adaptive Tabu Search for course timetabling*, European Journal of Operational Research, 2010.